
WEB CRAWLERS AND PAGERANK

Content

- **What is a web crawler?**
- Why is web crawler required?
- How does web crawler work?
- Crawling strategies
 - Breadth first search traversal
 - Depth first search traversal
- Architecture of web crawler
- Crawling policies

WEB CRAWLERS

- The process or program used by search engines to download pages from the web for later processing by a search engine that will index the downloaded pages to provide fast searches.
- A program or automated script which browses the World Wide Web in a methodical, automated manner
- also known as web spiders and web robots.
- less used names- ants, bots and worms.

Content

- What is a web crawler?
- **Why is web crawler required?**
- How does web crawler work?
- Crawling strategies

Breadth first search traversal

depth first search traversal

- Architecture of web crawler
- Crawling policies
- Distributed crawling

WHY CRAWLERS?

- ❖ Internet has a wide expanse of Information.
- ❖ Finding relevant information requires an efficient mechanism.
- ❖ Web Crawlers provide that scope to the search engine.

Web [Images](#) [Maps](#) [News](#) [Orkut](#) [Books](#) [Gmail](#) [more ▾](#) [Web History](#) | [Search settings](#) | [Sign out](#)

Google [Advanced Search](#)

Search: ☒ the web ☐ pages from India

Web [+ Show options...](#) Results 1 - 10 of about **12,900,000** for web crawling. **0.15** seconds

[Web crawler](#) - Wikipedia, the free encyclopedia
A **Web crawler** is a computer program that browses the World Wide Web in a methodical, automated manner. Other terms for Web crawlers are ants, ...
[Crawling policies](#) - [Web crawler architectures](#) - [Crawler identification](#)
[en.wikipedia.org/wiki/Web_crawler](#) - 16 hours ago - [Cached](#) - [Similar](#)

[Distributed web crawling](#) - Wikipedia, the free encyclopedia
Distributed **web crawling** is a distributed computing technique whereby Internet search engines employ many computers to index the Internet via **web crawling**. ...
[en.wikipedia.org/wiki/Distributed_web_crawling](#) - [Cached](#) - [Similar](#)

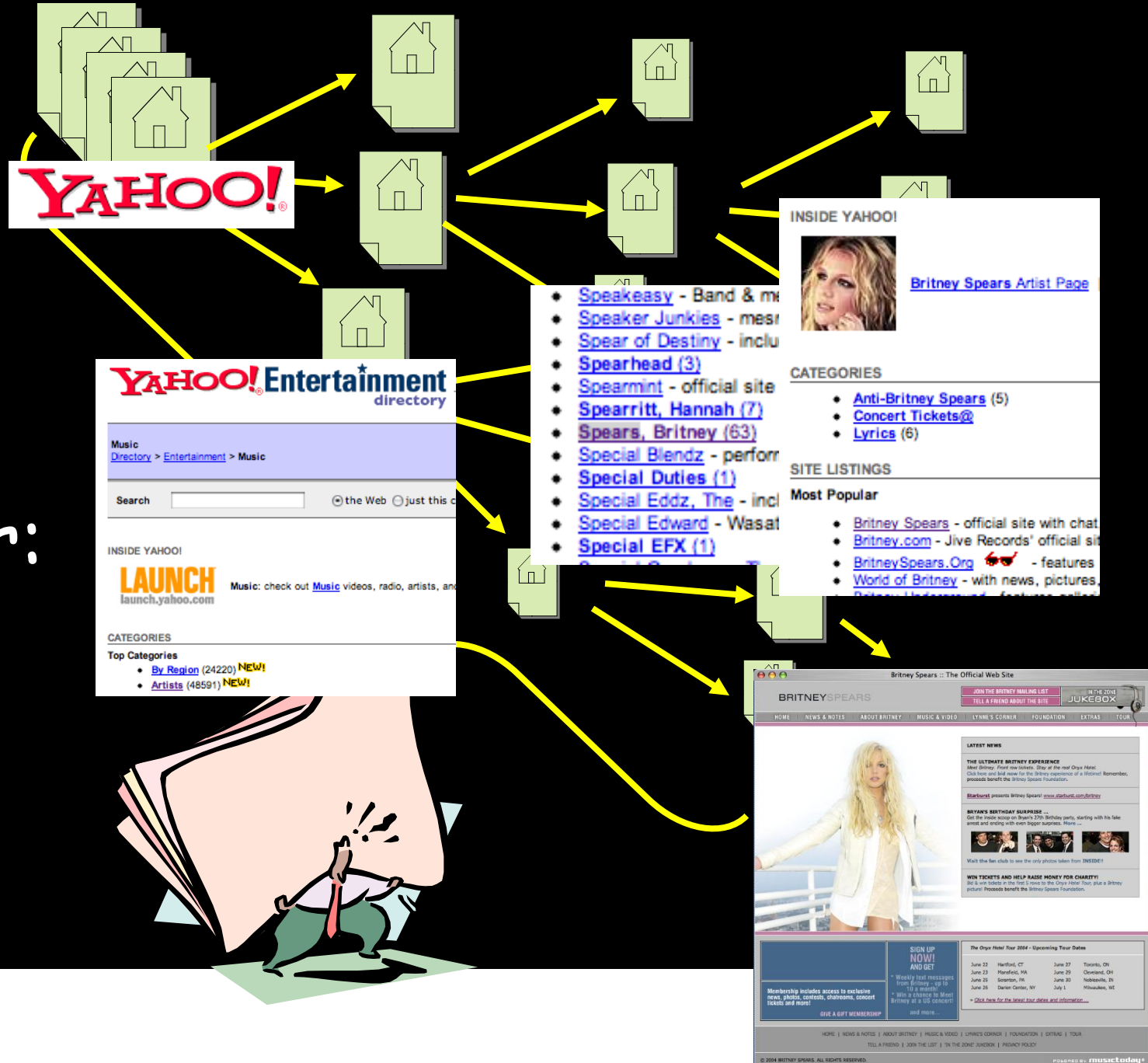
[What is a Web Crawler?](#)
Brief and Straightforward Guide: What is a **Web Crawler**?
[www.wisegeek.com/what-is-a-web-crawler.htm](#) - [Cached](#) - [Similar](#)

[Image results for web crawling](#) - [Report images](#)

[Video results for web crawling](#)

Crawler:
basic
idea



New url's can be specified here. This is google's web Crawler.



For Site Owners: Submit your site

[Home](#)

[All About Google](#)

[Advertise with Us](#)

[Search Solutions](#)

[Webmaster Info](#)

▶ [Submit Your Site](#)

Find on this site:

Search

Share your place on the net with us.

We add and update new sites to our index each time we crawl the web, and we invite you to submit your URL here. We do not add all submitted URLs to our index, and we cannot make any predictions or guarantees about when or if they will appear.

Please enter your full URL, including the `http://` prefix. For example: `http://www.google.com/`. You may also add comments or keywords that describe the content of your page. These are used only for our information and do not affect how your page is indexed or used by Google.

Please note: Only the top-level page from a host is necessary; you do not need to submit each individual page. Our crawler, Googlebot, will be able to find the rest. Google updates its index on a regular basis, so updated or outdated link submissions are not necessary. Dead links will 'fade out' of our index on our next crawl when we update our entire index.

URL:

Comments:

Add URL

Other Options

Instant Ads on Google

Create your own targeted ads using [AdWords](#). With credit card payment, you can see your ad on Google today.

Google AdSense for Web Publishers

Publish ads that match your content, help visitors find related products and services — and maximize your ad revenue. [Learn more.](#)

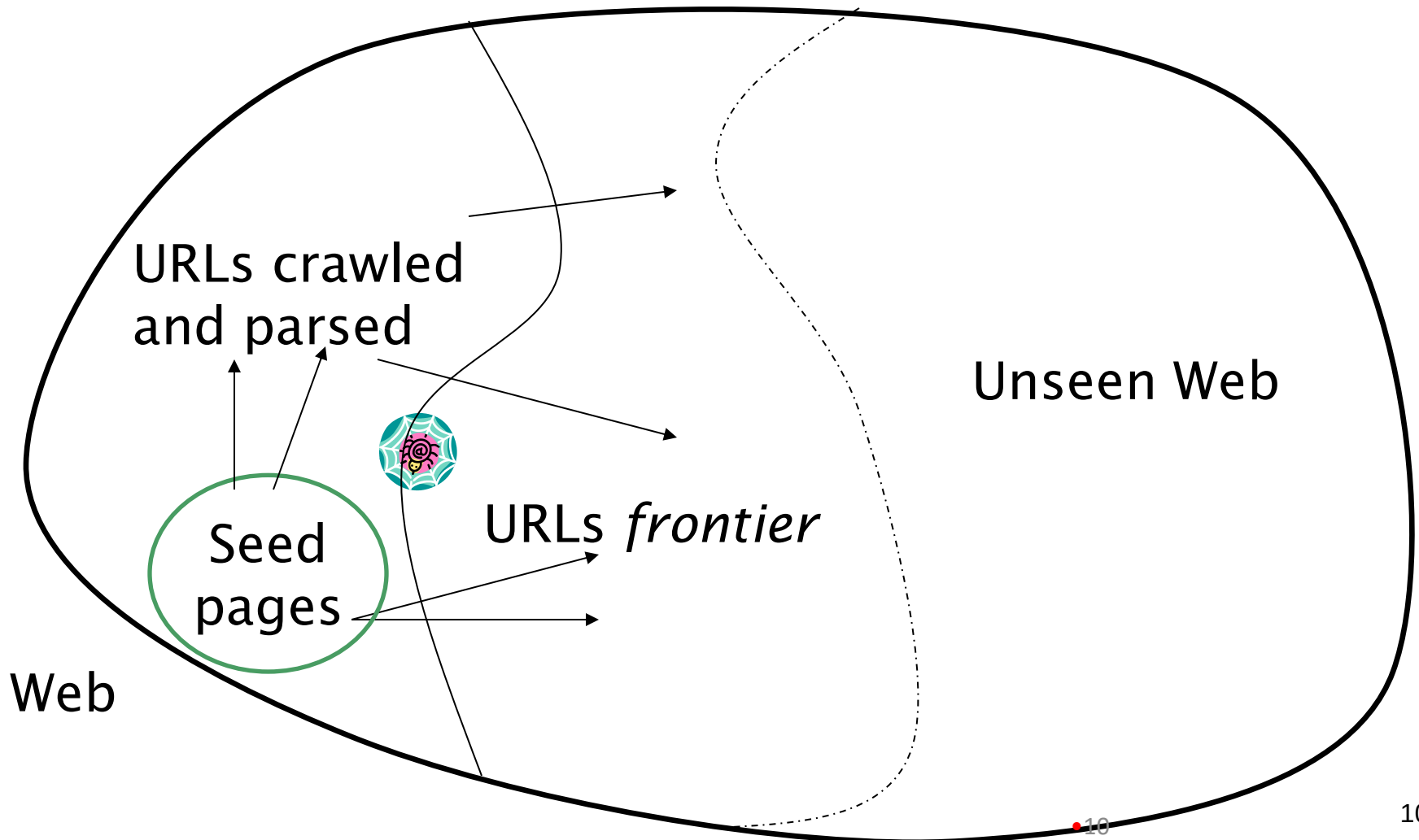
Content

- What is a web crawler?
- Why is web crawler required?
- **How does web crawler work?**
- Crawling strategies
 - Breadth first search traversal
 - depth first search traversal
- Architecture of web crawler
- Crawling policies
- Distributed crawling

How does web crawler work?

- It starts with a list of URLs to visit, called the **seeds**. As the crawler visits these URLs, it identifies all the hyperlinks in the page and adds them to the list of visited URLs, called the **crawl frontier or URL frontier**.
- URLs from the frontier are recursively visited according to a set of policies.

Crawling picture



Crawling Algorithm

Initialize queue (Q) with initial set of known URL's.

Until Q empty or page or time limit exhausted:

- Pop URL, L, from front of Q.

- If L is not an HTML page (.gif, .jpeg, .ps, .pdf, .ppt...) exit loop.

- If already visited L, continue loop(get next url).

- Download page, P, for L.

- If cannot download P (e.g. 404 error, robot excluded) exit loop, else.

- Index P (e.g. add to inverted index).

- Parse P to obtain list of new links N.

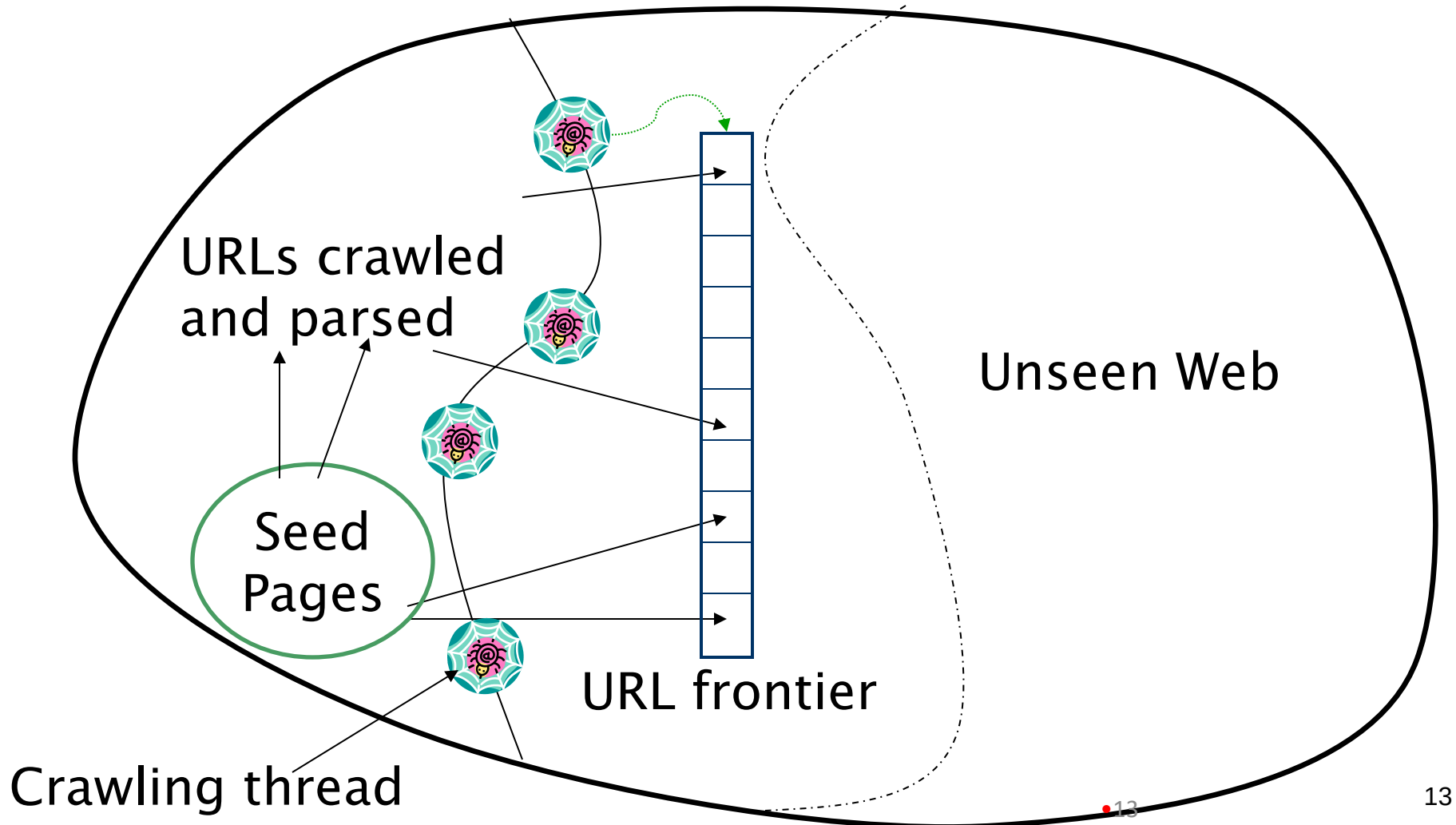
- Append N to the end of Q.

CRAWLING STRATEGIES

Given any graph and a set of seeds at which to start, the graph can be traversed using the algorithm

1. Put all the given seeds into the queue;
2. Prepare to keep a list of “visited” nodes (initially empty);
3. As long as the queue is not empty:
 - a. Remove the first node from the queue;
 - b. Append that node to the list of “visited” nodes
 - c. For each edge starting at that node:
 - i. If the node at the end of the edge already appears on the list of “visited” nodes or it is already in the queue, then do nothing more with that edge;
 - ii. Otherwise, append the node at the end of the edge to the end of the queue.

Crawling picture



Content

- What is a web crawler?
- Why is web crawler required?
- How does web crawler work?
- Crawling strategies

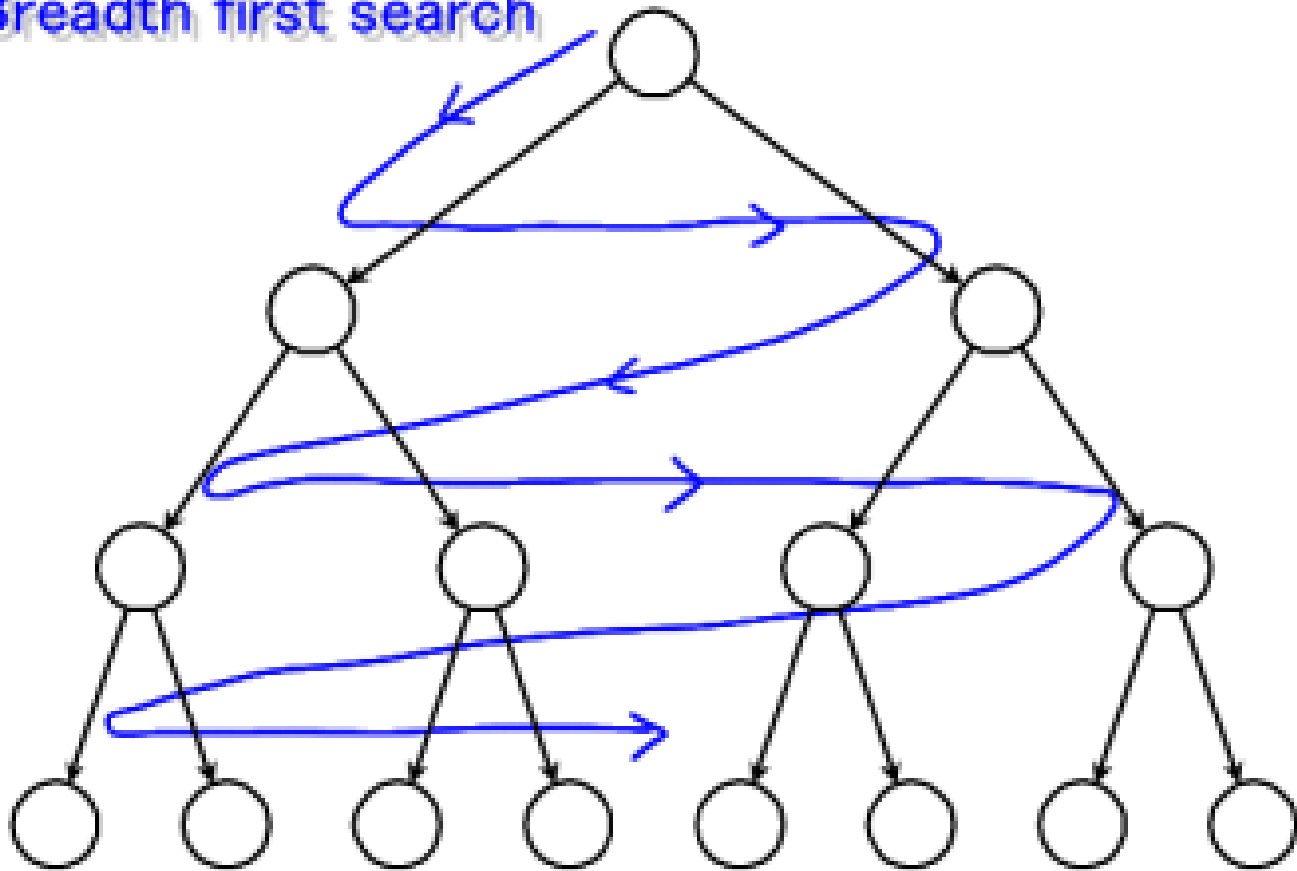
Breadth first search traversal

Depth first search traversal

- Architecture of web crawler
- Crawling policies

Breadth First Crawlers

Breadth first search



Content

- What is a web crawler?
- Why is web crawler required?
- How does web crawler work?
- Crawling strategies

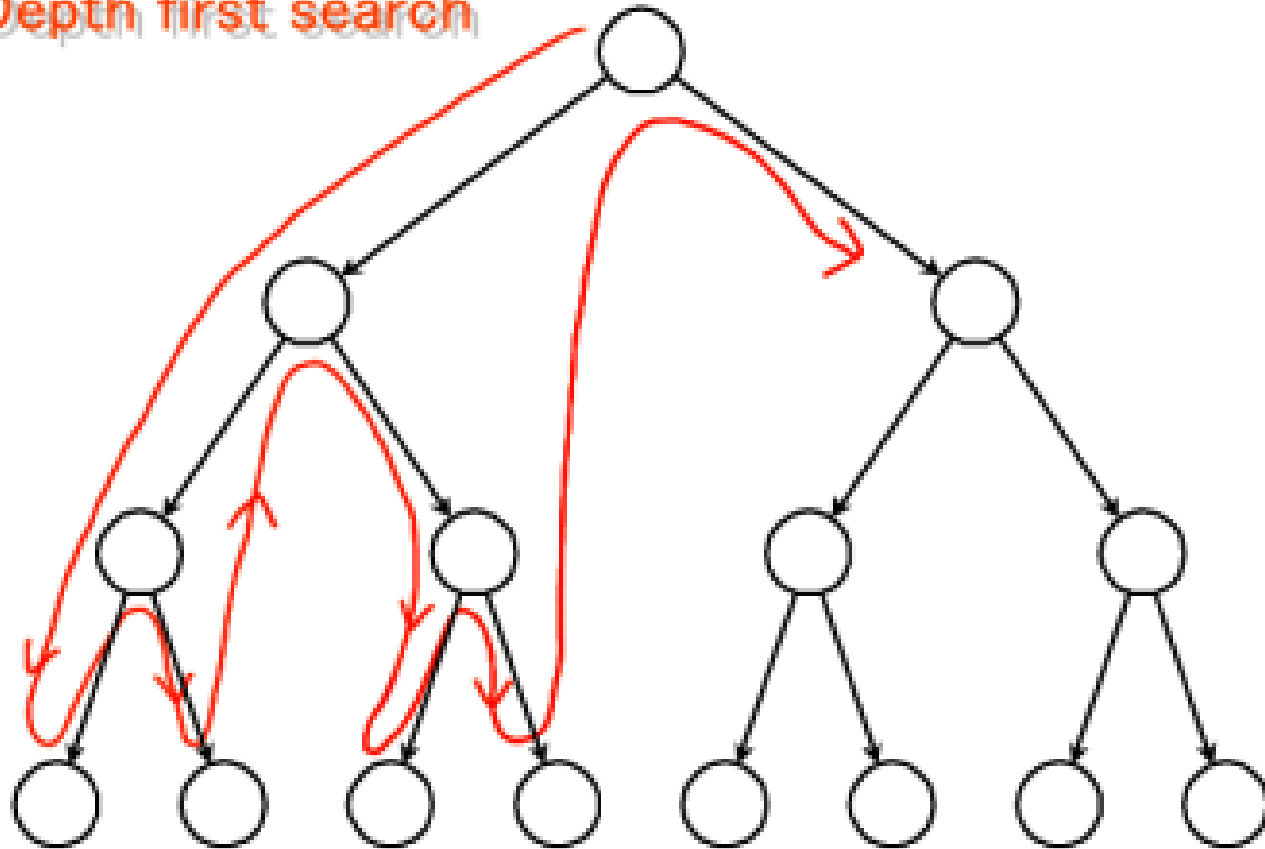
Breadth first search traversal

Depth first search traversal

- Architecture of web crawler
- Crawling policies

Depth first traversal

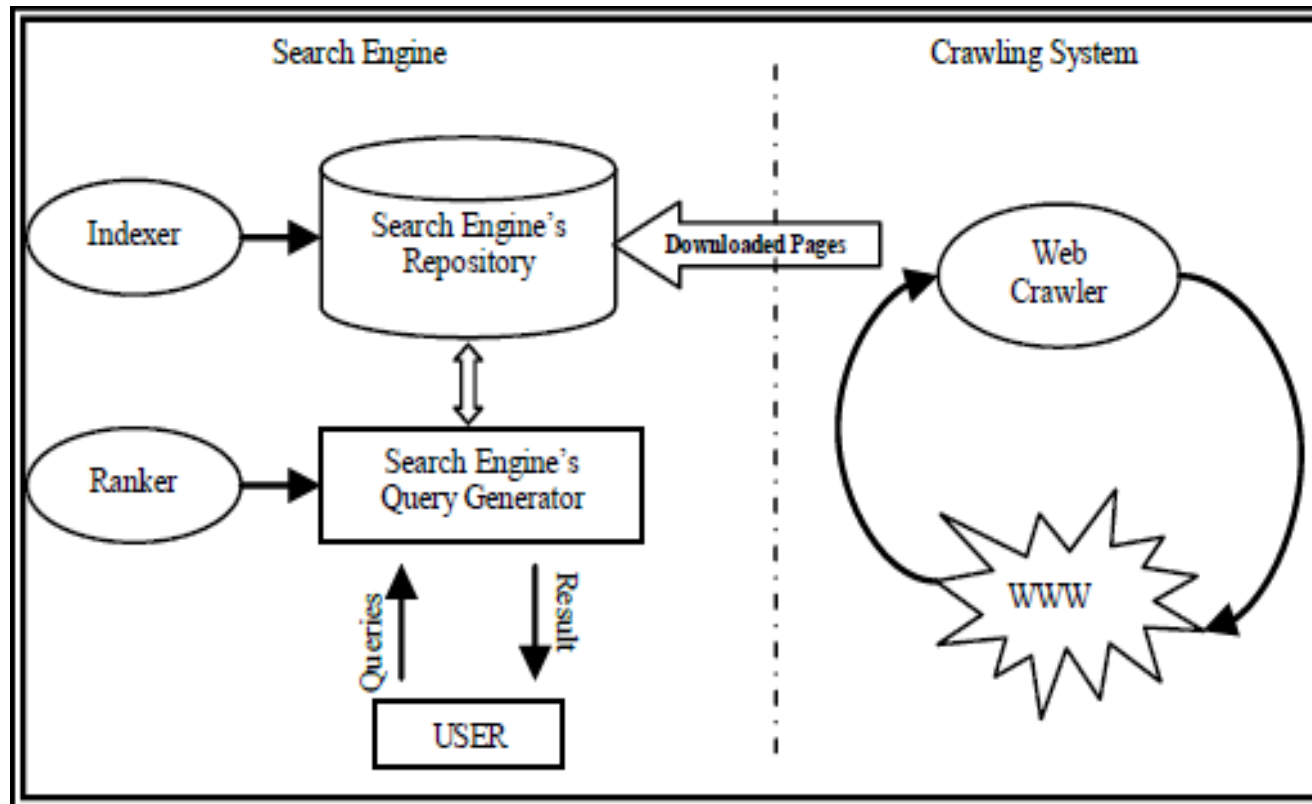
Depth first search



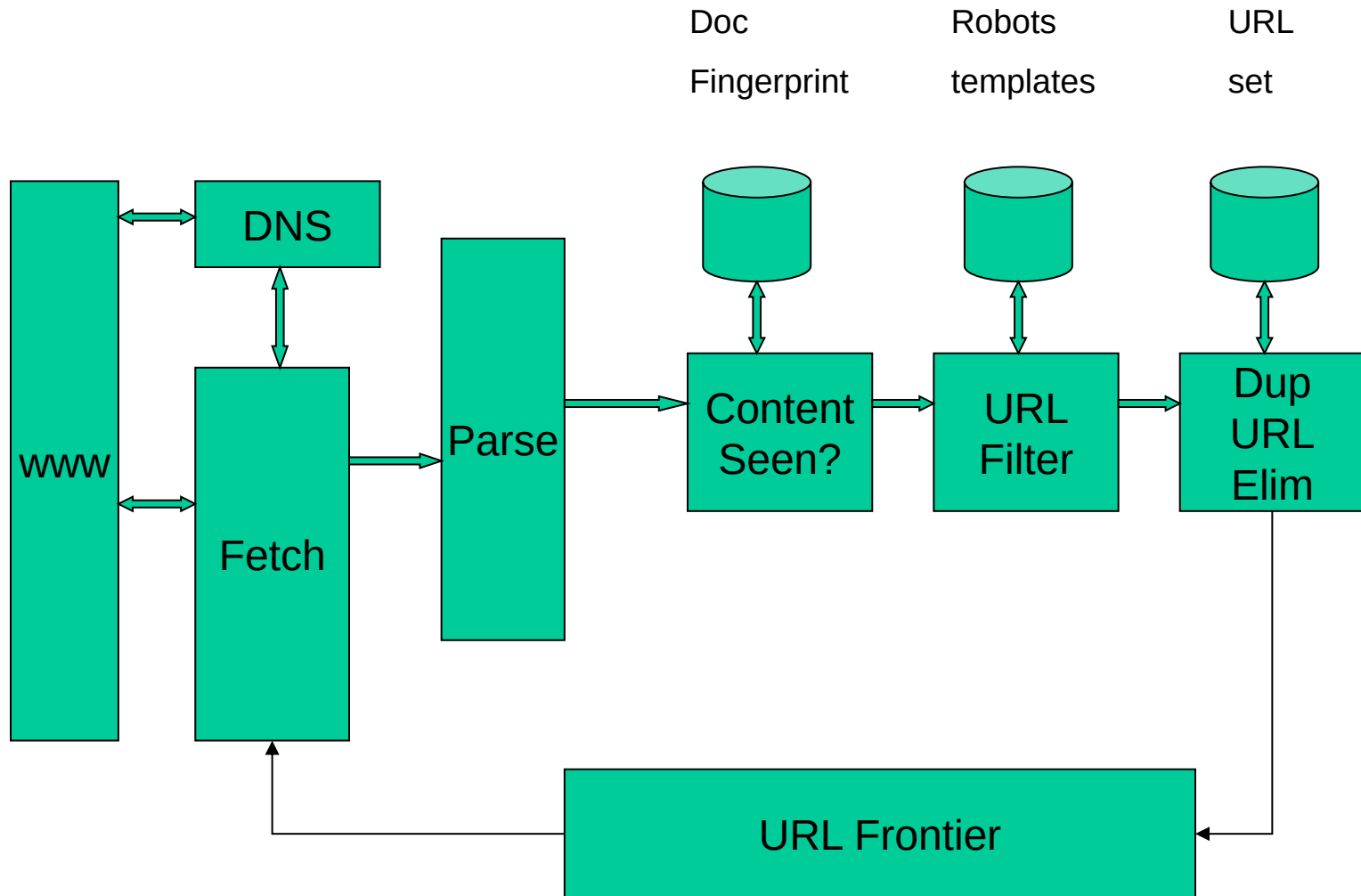
content

- What is a web crawler?
- Why is web crawler required?
- How does web crawler work?
- Crawling strategies
 - Breadth first search traversal
 - Depth first search traversal
- **Architecture of web crawler**
- Crawling policies

Architecture of search engine



ARCHITECTURE OF crawler



Architecture

- ❖ **URL Frontier:** containing URLs yet to be fetches in the current crawl. At first, a seed set is stored in URL Frontier, and a crawler begins by taking a URL from the seed set.
- ❖ **DNS:** domain name service resolution. Look up IP address for domain names.
- ❖ **Fetch:** generally use the http protocol to fetch the URL.
- ❖ **Parse:** the page is parsed. Texts (images, videos, and etc.) and Links are extracted.
- ❖ **Content Seen?:** test whether a web page with the same content has already been seen at another URL. Need to develop a way to measure the fingerprint of a web page.

Architecture

- ❖ **URL Filter:**

- Whether the extracted URL should be excluded from the frontier (robots.txt).

- ❖ **Dup URL Elim:** the URL is checked for duplicate elimination.

Content

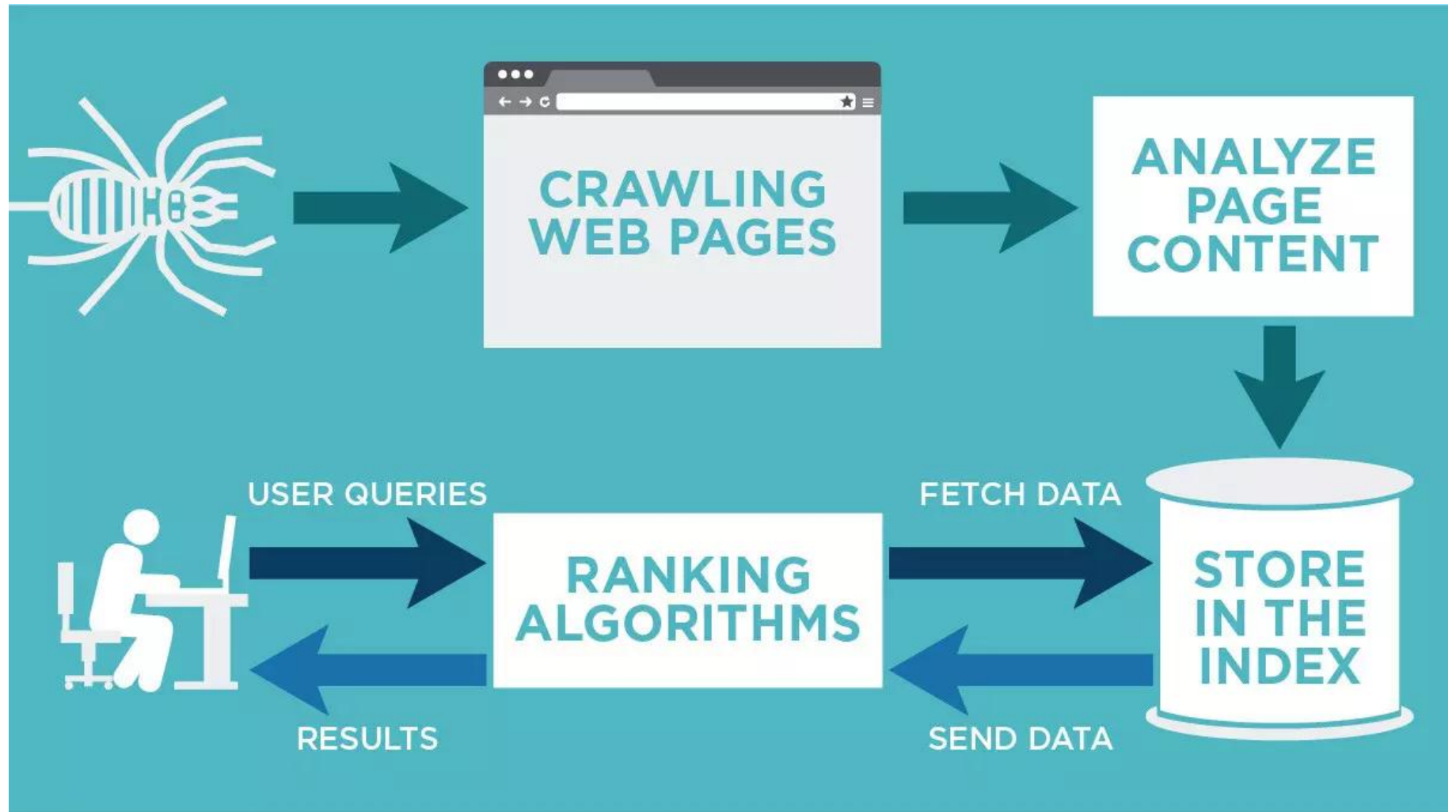
- What is a web crawler?
- Why is web crawler required?
- How does web crawler work?
- Crawling strategies
 - Breadth first search traversal
 - Depth first search traversal
- Architecture of web crawler
- **Crawling policies**

Crawling Policies

- Selection Policy that states which pages to download.
- Re-visit Policy that states when to check for changes to the pages.
- Politeness Policy that states how to avoid overloading Web sites.
- Parallelization Policy that states how to coordinate distributed Web crawlers.

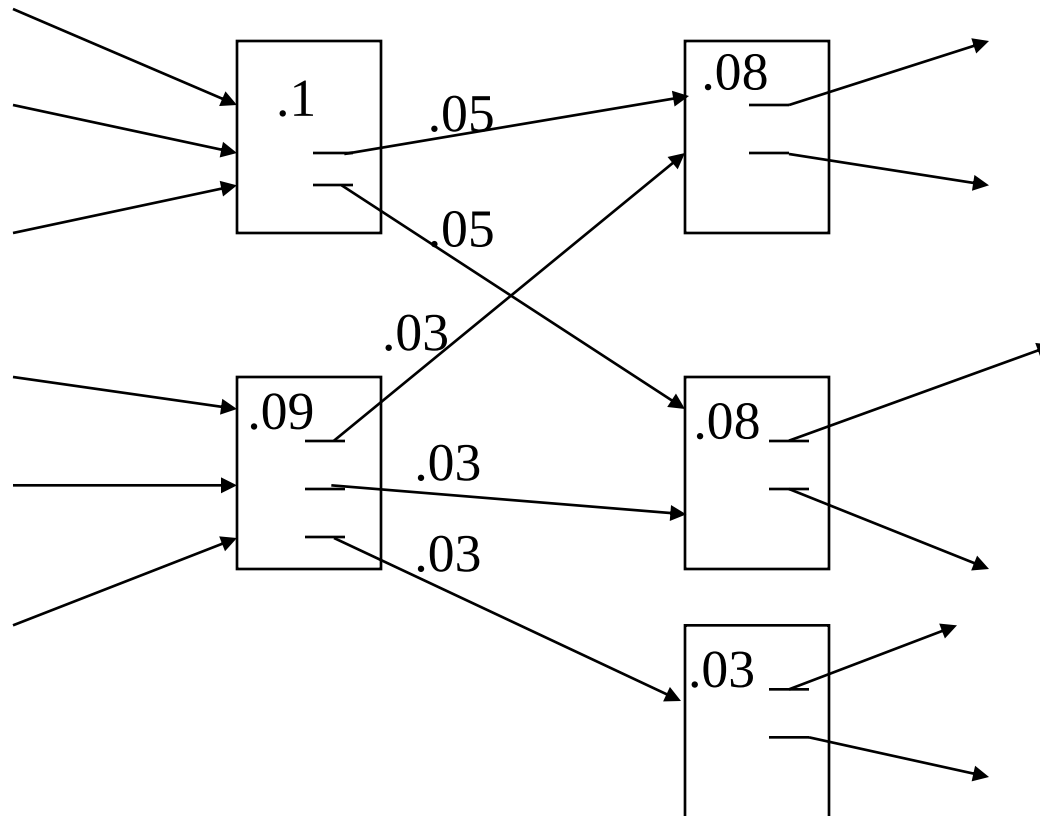
EXAMPLES

- **Yahoo! Slurp**: Yahoo Search crawler.
- **Msnbot**: Microsoft's Bing web crawler.
- **Googlebot** : Google's web crawler.
- **WebCrawler** : Used to build the first publicly-available full-text index of a subset of the Web.
- **World Wide Web Worm** : Used to build a simple index of document titles and URLs.
- **Web Fountain**: Distributed, modular crawler written in C++.
- **Slug**: Semantic web crawler



Initial PageRank Idea

- Can view it as a process of PageRank “flowing” from pages to the pages they cite.



Google Page Rank Algorithm

- Ranks Pages based on the number of other pages that link to it.
- In short PageRank is a “vote”, by all the other pages on the Web, about how important a page is.
- A link to a page counts as a vote of support
- $PR(A) = (1-d) + d(PR(T1)/C(T1) + \dots + PR(Tn)/C(Tn))$

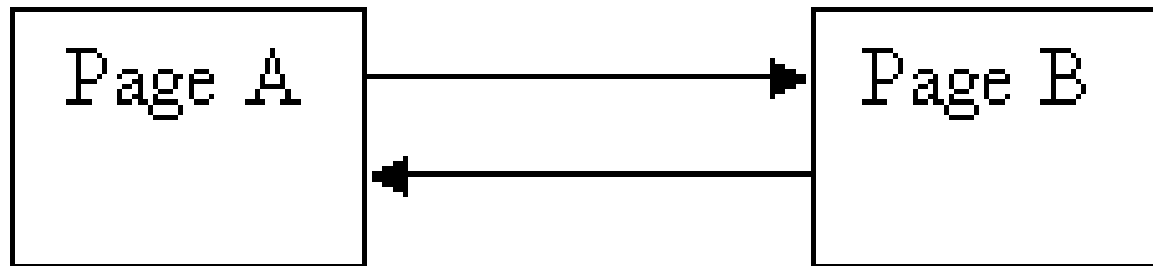
PageRank Algorithm

$$\text{PR}(A) = (1-d) + d \left(\frac{\text{PR}(T1)}{C(T1)} + \dots + \frac{\text{PR}(Tn)}{C(Tn)} \right)$$

Where:

- $\text{PR}(A)$ is the PageRank of page A ,
 - $\text{PR}(T_i)$ is the PageRank of pages T_i which link to page A ,
 - $C(T_i)$ is the number of outbound links on page T_i
- d is a damping factor which can be set between 0 and 1.

Simple Example



- Each page has one outgoing link.
- So that means $C(A) = 1$ and $C(B) = 1$.

We don't know what their PR should be to begin with, so we will just guess 1 as a safe random number.

- d (damping factor) = 0.85
- $PR(A) = (1 - d) + d(PR(B)/1)$
- $PR(B) = (1 - d) + d(PR(A)/1)$

i.e.

- $PR(A) = 0.15 + 0.85 * 1 = 1$
- $PR(B) = 0.15 + 0.85 * 1 = 1$

Let's Do It Again with Another Number.
Let's try 0 and re-calculate...

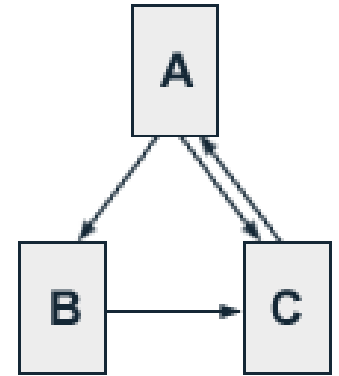
- $PR(A) = 0.15 + 0.85 * 0$
 $= 0.15$
 $PR(B) = 0.15 + 0.85 * 0.15$
 $= 0.2775$
- Now we have calculated a “next best guess” so we just plug it in the equation again...
- $PR(A) = 0.15 + 0.85 * 0.2775 = 0.385875$
- $PR(B) = 0.15 + 0.85 * 0.385875 = 0.47799375$

And again...

- $PR(A) = 0.15 + 0.85 * 0.47799375 = 0.5562946875$
- $PR(B) = 0.15 + 0.85 * 0.5562946875 = 0.622850484375$

The Characteristics of PageRank

- We regard a small web consisting of three pages A, B and C, whereby page A links to the pages B and C, page B links to page C and page C links to page A. The damping factor d is usually set to 0.85, but to keep the calculation simple we set it to 0.5.



$$PR(A) = 0.5 + 0.5 PR(C)$$

$$PR(B) = 0.5 + 0.5 (PR(A) / 2)$$

$$PR(C) = 0.5 + 0.5 (PR(A) / 2 + PR(B))$$

We get the following PageRank values for the single pages:

$$PR(A) = 1.07692308$$

$$PR(B) = 0.76923077$$

$$PR(C) = 1.15384615$$

The sum of all pages' PageRanks is 3 and thus equals the total number of web pages.

Example, whereby each page is assigned a starting PageRank value of 1

Iteration	PR(A)	PR(B)	PR(C)
0	1	1	1
1	1	0.75	1.125
2	1.0625	0.765625	1.1484375
3	1.07421875	0.76855469	1.15283203
4	1.07641602	0.76910400	1.15365601
5	1.07682800	0.76920700	1.15381050
6	1.07690525	0.76922631	1.15383947
7	1.07691973	0.76922993	1.15384490
8	1.07692245	0.76923061	1.15384592
9	1.07692296	0.76923074	1.15384611
10	1.07692305	0.76923076	1.15384615
11	1.07692307	0.76923077	1.15384615
12	1.07692308	0.76923077	1.15384615

Scraping

```
In [ ]: pip install requests
```

```
In [ ]: pip install beautifulsoup4
```

```
In [3]: from bs4 import BeautifulSoup
import requests
url="http://stackpython.co/courses"
res=requests.get(url)
res.encoding="utf-8"
print(res)
```

```
<Response [200]>
```

```
In [7]: if res.status_code==200:
        print("Successful")
        elif res.status_code==404:
        print("Error 404 page not found")
        else:
        print("Not both 200 and 404")
```

```
Successful
```

```
In [10]: soup=BeautifulSoup(res.text,'html.parser')
print(soup)
```

```
<!DOCTYPE html>

<html lang="en">
<head>
<meta charset="utf-8"/>
<title>Python Courses - คอร์สเรียนไพธอน | STACKPYTHON</title>
<meta content="เรามีคอร์สเรียนต่าง ๆ ที่เกี่ยวกับภาษาไพธอน ไม่ว่าจะเป็น Django, Flask, Machine Learning และเทคโนโลยีที่เกี่ยวข้อง คัดสรรและตั้งใจทำอย่างดีทุกคอร์ส ในราคาที่ถูกทุกคนสามารถเข้าถึงได้ คอร์สเรียนไพธอนที่ถูกและดี ต้องเรียนที่ stackpython" name="description"/>
<meta content="เรียนไพธอน, เรียน python, python courses, python programming, learn python, web development, django, flask, machine learning" name="keywords"/>
<meta content="STACKPYTHON" property="og:site_name"/>
<link href="https://stackpython.co/courses/" rel="canonical"/>
<meta content="width=device-width, initial-scale=1.0" name="viewport"/>
<meta content="#b1040e" name="theme-color">
<link href="/static/blog/img/stp.png" rel="icon"/>
<link href="/static/blog/img/stp.png" rel="apple-touch-icon"/>
<link href="https://fonts.googleapis.com/css?family=Poppins:300,400,500,600,700" rel="stylesheet"/>
<link crossorigin="anonymous" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0/css/bootstrap.min.css" integrity="sha
```

Scraping

In [11]: `print(soup.prettify())`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8"/>
  <title>
    Python Courses - คอร์สเรียนไพธอน | STACKPYTHON
  </title>
  <meta content="เรามีคอร์สเรียนต่าง ๆ ที่เกี่ยวกับภาษาไพธอน ไม่ว่าจะเป็น Django, Flask, Machine Learning และเทคโนโลยีที่เกี่ยวข้อง คัดสรรและ
  ตั้งใจทำอย่างดีทุกคอร์ส ในราคาที่ทุกคนสามารถเข้าถึงได้ คอร์สเรียนไพธอนที่ถูกและดี ต้องเรียนที่ stackpython" name="description"/>
  <meta content="เรียนไพธอน, เรียน python, python courses, python programming, learn python, web development, django, flask,
  machine learning" name="keywords"/>
  <meta content="STACKPYTHON" property="og:site_name"/>
  <link href="https://stackpython.co/courses/" rel="canonical"/>
  <meta content="width=device-width, initial-scale=1.0" name="viewport"/>
  <meta content="#b1040e" name="theme-color">
  <link href="/static/blog/img/stp.png" rel="icon"/>
  <link href="/static/blog/img/stp.png" rel="apple-touch-icon"/>
  <link href="https://fonts.googleapis.com/css?family=Poppins:300,400,500,600,700" rel="stylesheet"/>
  <link crossorigin="anonymous" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0/css/bootstrap.min.css" integrity
```

In [13]: `soup.title`

Out[13]: `<title>Python Courses - คอร์สเรียนไพธอน | STACKPYTHON</title>`

In [14]: `soup.h1`

Out[14]: `<h1 class="title-single title-a">Python Courses - คอร์สเรียนไพธอน</h1>`

In [15]: `soup.p`

```
<p style="font-family: Verdana, Geneva, Tahoma, sans-serif; font-size: 18px; ">พื้นฐานการเรียนรู้ไพธอนครบจบในคอร์ส Introduction to
Python Programming คอร์สนี้คอร์สเดียว

    <div class="row">
      <h4>👋 </h4>
    </div>
    <h4> <i aria-hidden="true" class="fa fa-clock-o"></i> 10 ชั่วโมง</h4>
  </p>
```

In [16]: `soup.title.string`

Out[16]: `'Python Courses - คอร์สเรียนไพธอน | STACKPYTHON'`

```
In [18]: soup.find_all('a')
```

```
Out[18]: <a class="navbar-brand text-center" href="/">STACKPYTHON</a>,  
  <a class="nav-link" href="/">Home</a>,  
  <a class="nav-link" href="/about">About</a>,  
  <a class="nav-link" href="/authors">Authors</a>,  
  <a class="nav-link" href="/contact-us">Contact</a>,  
  <a aria-expanded="false" aria-haspopup="true" class="nav-link dropdown-toggle" data-toggle="dropdown" href="/tutorials"  
id="navbarDropdown" role="button">  
    Blog  
  </a>,  
  <a class="dropdown-item" href="/tutorials">Tutorials</a>,  
  <a class="dropdown-item" href="/programming-news">News</a>,  
  <a class="card mb-5 hover-effect" href="/course/introduction-to-python-programming" style="max-width: 900px; border-widt  
h: 0px; border-radius: 10px; background-color: white;">  
    <div class="row no-gutters">  
      <div class="col-md-6">  
          
      </div>  
      <div class="col-md-6">
```

```
In [20]: courses=soup.find_all('h2')
         print(courses)
```

```
[<h2 class="title-single title-a" style="font-size: 34px;">Introduction to Python Programming
    </h2>, <h2 class="title-single title-a" style="font-size: 34px;">Python Web Development with Flask
    </h2>, <h2 class="title-single title-a" style="font-size: 34px;">Full Stack Vue.js with Flask (REST Api)
    </h2>, <h2 class="title-single title-a" style="font-size: 34px;">Python Web Development with Django
    </h2>, <h2 class="title-single title-a" style="font-size: 34px;">Python Programming & Web Development 2021
Full Courses
    </h2>, <h2 class="title-single title-a" style="font-size: 34px;">Python Line Chatbot
    </h2>, <h2 class="title-single title-a" style="font-size: 34px;">Python Pandas
    </h2>, <h2 class="title-single title-a" style="font-size: 34px;">Basic Machine Learning with Python
    </h2>, <h2 class="title-single title-a" style="font-size: 34px;">Django REST Framework
    </h2>, <h2 class="title-single title-a" style="font-size: 34px;">Django Wagtail CMS
    </h2>]
```

Scraping

```
In [20]: > courses=soup.find_all('h2')
print(courses)

[<h2 class="title-single title-a" style="font-size: 34px;">Introduction to Python Programming
  </h2>, <h2 class="title-single title-a" style="font-size: 34px;">Python Web Development with Flask
  </h2>, <h2 class="title-single title-a" style="font-size: 34px;">Full Stack Vue.js with Flask (REST Api)
  </h2>, <h2 class="title-single title-a" style="font-size: 34px;">Python Web Development with Django
  </h2>, <h2 class="title-single title-a" style="font-size: 34px;">Python Programming & Web Development 2021
Full Courses
  </h2>, <h2 class="title-single title-a" style="font-size: 34px;">Python Line Chatbot
  </h2>, <h2 class="title-single title-a" style="font-size: 34px;">Python Pandas
  </h2>, <h2 class="title-single title-a" style="font-size: 34px;">Basic Machine Learning with Python
  </h2>, <h2 class="title-single title-a" style="font-size: 34px;">Django REST Framework
  </h2>, <h2 class="title-single title-a" style="font-size: 34px;">Django Wagtail CMS
  </h2>]
```

```
In [25]: > course_list=[]
for course in courses:
    obj=course.string
    course_list.append(obj)

['Introduction to Python Programming \n', 'Python Web Development with Flask \n', 'Full Stack
Vue.js with Flask (REST Api) \n', 'Python Web Development with Django \n', 'Python Program
ming & Web Development 2021 Full Courses \n', 'Python Line Chatbot \n', 'Python Pandas \n',
'Basic Machine Learning with Python \n', 'Django REST Framework \n', 'Django Wagtail CMS
\n']
```

```
In [29]: > import csv
csv_col=[['title'],[course_list]]
f=open('course_titles.csv','w')
with f:
    writer=csv.writer(f)
    for row in csv_col:
        writer.writerow(row)
```

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import pandas as pd
import time

# 📌 ตั้งค่า ChromeDriver (เปลี่ยนเป็น path ของคุณ)
chrome_driver_path = r"C:\Users\Asus\Documents\chrome-win64\chromedriver-win64\chromedriver"
service = Service(chrome_driver_path)
driver = webdriver.Chrome(service=service)

# 🔍 เปิดเว็บหนังสือ
driver.get("https://books.toscrape.com/")
time.sleep(3)

# ✅ ใช้ WebDriverWait เพื่อรอให้โหลดเสร็จ
try:
    WebDriverWait(driver, 10).until(
        EC.presence_of_element_located((By.CLASS_NAME, "product_pod"))
    )
except:
    print("❌ โหลดเว็บไม่สำเร็จ")
    driver.quit()
    exit()

# 🔍 ดึงข้อมูลหนังสือ
book_list = []
books = driver.find_elements(By.CLASS_NAME, "product_pod")
```

```

for book in books[:10]: # ดึงแค่ 10 เล่มแรก
    try:
        title = book.find_element(By.TAG_NAME, "h3").text
        price = book.find_element(By.CLASS_NAME, "price_color").text
        link = book.find_element(By.TAG_NAME, "a").get_attribute("href")

        book_list.append({"ชื่อหนังสือ": title, "ราคา": price, "ลิงก์": link})
    except:
        continue

# 📌 บันทึกข้อมูลเป็น CSV
df = pd.DataFrame(book_list)
df.to_csv("books.csv", index=False, encoding="utf-8-sig")

# ✅ ปิดเบราว์เซอร์
driver.quit()

# 🚩 แสดงตัวอย่างข้อมูลที่ดึงมา
df.head()

```

Out[1]:

	ชื่อหนังสือ	ราคา	ลิงก์
0	A Light in the ...	£51.77	https://books.toscrape.com/catalogue/a-light-i...
1	Tipping the Velvet	£53.74	https://books.toscrape.com/catalogue/tipping-t...
2	Soumission	£50.10	https://books.toscrape.com/catalogue/soumissio...
3	Sharp Objects	£47.82	https://books.toscrape.com/catalogue/sharp-obj...
4	Sapiens: A Brief History ...	£54.23	https://books.toscrape.com/catalogue/sapiens-a...